

Machine-Learning-fundations

This repository is for the Machine Learning foundations at my university. Here I will develop my first project for this course.

Name of student: Laura Somogyi

Course: Machine Learning foundation

Semester: 2025-2026 2nd semester

AI disclaimer: I have only used generative AI for some coding parts, since my coding skills are not the best yet. However, I have done the explanations and analysis myself WITHOUT the use of any external help.

Task 1 -> Identifying the Prediction Target

- Inspect the dataset and identify which column should be treated as the target variable for this assignment.
- Justify why this column represents the appropriate prediction objective in the context of the marketing campaign
- Identify at least two other variables that could superficially appear to be valid targets and explain why they should not be treated as the prediction objective

My answers

* The column that should be treated as a target variable is the "y" column since it addresses the question whether the client will subscribe to the term deposit or not.

* It is appropriate prediction objective in the context of the marketing campaign because as I mentioned above the subscribed or not will be marked here and this decides whether the bank gets a share from penalties or not or they manage to get a client or not.

* potential other target variables

* "poutcome" column

* This feature represents whether the previous campaign was successful, failed or none-existent.

* This could be a reasonable if we had information about the previous interviews like duration and when it took place. However the reason I did not pick this is it may be correlated with the one included in this dataset. So the outcome may include the effect of both.

* "balance" column

* It could be a possible target because it keeps track of the average yearly balance on the term deposit account and this can be predicted from many variables.

* However I will not treat this as a prediction objective because it is not available at the time when we want to predict whether the client will commit to the creation of the account or not. Until then the balance is either 0 or missing or simply

just unknown

```
#here I import the necessary libraries
#later I will add more libraries if I need to
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Task 2 Task Ordering

- Lecture material:

* Lecture 2 (Data Splitting and Leakage), Lecture 5 (Preprocessing), Lecture 9 (ML Pipeline)

- Determine the correct order in which the data preparation tasks in this assignment should be performed
- Provide a structured justification for your chosen order
- For each step in your proposed sequence, explain:

* what information is allowed to be used at that stage;

* what information must not be used;

* what type of data leakage could occur if the order were changed

- discuss at least one example of an incorrect ordering and explain the consequences it would have on model evaluation

My answer

For the order I have decided on the one which can be seen in this jupyter notebook for the following reasons:

* First I chose to identify the prediction target, which gave me an idea which way should I start my assignment. Then I chose task ordering, to structure my tasks according to the target variable and what seemed reasonable to work with the data.

* Here every information is allowed to be used

* Then the reason I chose data exploration as 3rd is to verify whether I can go on with my data to further train my model.

* Here every information is allowed to be used

* I chose managing missing values as 4th is because in my opinion, handling missingness early would make sure that the model can learn earlier from the missing values

* Here every information is allowed to be used

* I put encoding categorical variables 5th because it is better if I have it before feature selection, so everything represented as numbers to make sure the model does not waste energy on trying to figure out text input.

* Here only the categorical variables should be used only

* I put feature encoding 6th because again it is important to have everything in a matching format which is integer type.

* Here those information can be used that is not in some numerical format

* data splitting

* after everything is ready I can start to work on the data because it is all in the same datatype and easier to handle

* Here the encoded informations can be used

- * class imbalance
- * after all the work done, I have to also check this
- * Here only the classes can be used
- * feature scaling
- * after finishing the project I realised I should have put this after feature encoding.
- * only encoded features can be used
- * Finally, training the model after everything is ready beforehand.
- * Here every information set so far can be used

Task 3: Data Loading and Exploration

Lecture material: Lecture 1 (Problem Formulation), Lecture 2 (Data Inspection and EDA).

- Load the dataset into a Pandas DataFrame.
- Inspect the structure of the dataset: number of observations, number of features, data types, and basic summary statistics.
- Identify which variables are numerical and which are categorical

Analyze the distribution of the target variable and comment on potential class imbalance.

- Detect explicit and implicit missing values (e.g., special categories such as unknown).
- Visualize the distribution of at least:
 - two numerical variables; and
 - two categorical variables.
- Identify at least one variable that may require special consideration before modeling (e.g., due to distributional properties, extreme skewness, or availability at prediction time), and briefly justify your reasoning.

Note: Exploratory analysis is not a checklist of plots. Each visualization or statistic should support a specific observation or hypothesis about the data

My explanation:

- * numerical variables
- * age
- * default
- * balance
- * housing

- * loan
- * duration
- ÷ campaign
- * pdays
- * previous
- * categorycal variables
- * poutcome
- * contact
- * education
- * marital
- * job
- * date type variables
- * month
- * day_of_week

* distribution of the target variable and comment on pototential class imbalance

* By taking a look at the barblots in my code it is visible for me that 36,548 (88.7%) from the contacted people did not sign the contract with the bank.

* On the other side 4,640 (11,3%) of contacted people decided on signing up for the subscription offered by the bank.

* This implies a quite large class imbalance in the subcription acception rate. Seemingly the majority did not accept the contract and the rest did.

* It is very much a skewed distribution, more in detail right skewed.

* missing values

* explicit

* poutcome

* default

* implicit

* previous

* marital status

* education

* one variable that may require special consideration before modeling

* Maybe it is the job variable, because it very much effects whether a person is eligible for a contract or not. This is also corrolated with income since I assume the person has to pay for the contract and wage depends on the job type. Therefore there are income differences and other variables can be assumed from this. For example the person had a loan or whether the person defaulted or not.

* plot usage explanation

* For subscription distribution I used the barplot because it represents 2 values compared to each other very nicely. Also, I have included some numerical values and percentages so it provides those informations visually too.

* For age distribution I have used both gaussian kde plot which is a non-parametric, smooth, continuous curve used to visualize the probability density function (PDF) of a dataset and barplot. The barplot is because it represents the distribution including every age as a bar which holds a lot more information. On the other side the gaussian kde function is a little more simplified and shows us well the distribution of ages which is nearly normal but a bit more right skewed.

* For representing the job I used a barplot because it separates the distinct jobs and shows the number of people working in that job and also showing a distribution if we look at the whole diagram. Also for the relationship status is for the same reason that it is much more informative as a barplot than any other.

AI prompt disclaimer:

- prompt -> "How do I plot a binary target variable in python from a CSV file?"

* reply:

* Here's a complete guide to plotting a binary target variable:

* Count Plot (Most Common for Binary Targets)

```
sns.countplot(x='target', data=df)

plt.title('Distribution of Target Variable')

plt.xlabel('Class')

plt.ylabel('Count')

plt.show()
```

* Bar Chart with Percentages

```
counts = df['target'].value_counts()

percentages = df['target'].value_counts(normalize=True) * 100
```

```
plt.bar(counts.index, counts.values, color=['steelblue', 'salmon'])

plt.title('Target Variable Distribution')

plt.xlabel('Class (0 = Negative, 1 = Positive)')

plt.ylabel('Count')
```

Add percentage labels on bars

```
for i, (count, pct) in enumerate(zip(counts.values, percentages.values)):

    plt.text(i, count + 5, f'{pct:.1f}%', ha='center', fontweight='bold')
```

```
plt.show()
```

* Pie Chart

```
df['target'].value_counts().plot(

    kind='pie',

    labels=['Negative (0)', 'Positive (1)'],

    autopct='%1.1f%%',

    colors=['steelblue', 'salmon']

)

plt.title('Target Variable Distribution')

plt.ylabel('')

plt.show()
```

* All plots together

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```

Count plot

```
sns.countplot(x='target', data=df, ax=axes[0])  
axes[0].set_title('Count Plot')
```

Bar chart

```
counts = df['target'].value_counts()  
axes[1].bar(counts.index, counts.values, color=['steelblue', 'salmon'])  
axes[1].set_title('Bar Chart')
```

Pie chart

```
df['target'].value_counts().plot(kind='pie', autopct='%1.1f%%', ax=axes[2])  
axes[2].set_title('Pie Chart')
```

```
plt.tight_layout()
```

```
plt.show()
```

* Install dependencies if needed: `pip install pandas matplotlib seaborn`

* Replace 'target' with your actual column name and 'your_file.csv' with your file path. The count plot or bar chart is generally recommended for binary targets as they make class imbalance immediately obvious.

- 2nd prompt -> "How do I print out more data from a CSV file in python?"

ANSWER I GOT

Here are the different ways to print more data from a CSV file:

1. Load the Data

```
pythonimport pandas as pd
```

```
df = pd.read_csv('your_file.csv')
```

2. Print More Rows

```
python# Default only shows 5 rows
```

```
print(df.head())
```

how more rows

```
print(df.head(20)) # first 20 rows
```

```
print(df.tail(20)) # last 20 rows
```

```
print(df.head(100)) # first 100 rows
```

3. Print All Rows & Columns (No Truncation)

```
pythonpd.set_option('display.max_rows', None) # show all rows
pd.set_option('display.max_columns', None) # show all columns
pd.set_option('display.width', None) # no line wrapping
pd.set_option('display.max_colwidth', None) # full column content
```

```
print(df)
```

4. Print Specific Columns

```
python# Single column
print(df['column_name'])
```

Multiple columns

```
print(df[['col1', 'col2', 'col3']])
```

5. Print Specific Rows

```
python# By index range
print(df[0:50]) # rows 0 to 49
```

By condition

```
print(df[df['age'] > 30])
```

By specific index

```
print(df.iloc[10]) # single row
print(df.iloc[10:20]) # row range
```

6. Useful Summary Prints

```
pythonprint(df.shape) # (rows, columns)
print(df.columns) # all column names
print(df.dtypes) # data types of each column
print(df.describe()) # stats summary (count, mean, std, etc.)
print(df.info()) # overview of the dataframe
print(df.nunique()) # unique values per column
print(df.isnull().sum()) # missing values per column
```

7. Reset Options Back to Default

```
pythonpd.reset_option('display.max_rows')
pd.reset_option('display.max_columns')
```

Quick Reference:

```
GoalCodeFirst N rowsdf.head(N)Last N rowsdf.tail(N)All rowspd.set_option('display.max_rows', None)Specific
```

```
columns=df[['col1', 'col2']]Filter rows=df[df['col'] > value]Summary stats=df.describe()
```

For your bank dataset specifically, since it uses semicolons as separators, remember to load it with:

```
pythondf = pd.read_csv('bank-additional-full.csv', sep=';')
```

```
bank_data = pd.read_csv('bank-additional-full.csv', sep=';') #loading the CSV file separated by commas
display(bank_data.head(100)) #printing out the first hundred rows of data
```

```

   age      job  marital      education  default housing loan  \
0   56  housemaid  married      basic.4y      no      no      no
1   57   services  married      high.school  unknown      no      no
2   37   services  married      high.school      no      yes      no
3   40    admin.  married      basic.6y      no      no      no
4   56   services  married      high.school      no      no      yes
..  ...      ...      ...      ...      ...      ...      ...
95  45   services  married  professional.course      no      yes      no
96  42  management  married  university.degree      no      no      no
97  53    admin.  divorced  university.degree  unknown      no      no
98  37  technician  single  professional.course      no      no      no
99  44  blue-collar  married      basic.6y      no      no      no

   contact month  day_of_week  ...  campaign  pdays  previous  poutcome  \
0  telephone  may      mon      ...      1      999      0  nonexistent
1  telephone  may      mon      ...      1      999      0  nonexistent
2  telephone  may      mon      ...      1      999      0  nonexistent
3  telephone  may      mon      ...      1      999      0  nonexistent
4  telephone  may      mon      ...      1      999      0  nonexistent
..      ...      ...      ...      ...      ...      ...      ...
95  telephone  may      mon      ...      1      999      0  nonexistent
96  telephone  may      mon      ...      1      999      0  nonexistent
97  telephone  may      mon      ...      1      999      0  nonexistent
98  telephone  may      mon      ...      1      999      0  nonexistent
99  telephone  may      mon      ...      1      999      0  nonexistent

   emp.var.rate  cons.price.idx  cons.conf.idx  euribor3m  nr.employed  y
0             1.1             93.994

```

```

#Plot of the target variable
#including the percentage values and count on top of the bars
counts = bank_data['y'].value_counts()
plt.bar(counts.index, counts.values, color=['#66c2a5', '#fc8d62'], width=0.5)
plt.title('Subscription Distribution')
plt.xlabel('Subscribed (yes/no)')
plt.ylabel('Count')
for p in plt.gca().patches:
    plt.gca().annotate(f'{p.get_height():.0f}\n({p.get_height()/len(bank_data)*100:.1f}%)',
                      (p.get_x() + p.get_width()/2, p.get_height()),
                      ha='center', va='bottom', fontweight='bold')
plt.show()

```

<Figure size 640x480 with 1 Axes>

```
# two numerical variable plot
```

```
# first choice is age
```

```
from scipy.stats import gaussian_kde
```

```

kde = gaussian_kde(bank_data['age'])
x = np.linspace(bank_data['age'].min(), bank_data['age'].max(), 200)
plt.plot(x, kde(x))
plt.title('Distribution of Age')
plt.xlabel('Age')
plt.ylabel('Density')
plt.show()
#I chose this specific plot type, because when I used barplots before, it was too dense

```

<Figure size 640x480 with 1 Axes>

```
#just to be sure that the plot is correct, I will also use a histogram, without any text on top of the bars, because it would be too dense
```



```
counts = bank_data['age'].value_counts()
plt.bar(counts.index, counts.values, color=['#66c2a5', '#fc8d62'], width=0.5)
plt.title('Age Distribution')
plt.xlabel('ages')
plt.ylabel('Count')
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
#second choice -> employment distribution
import seaborn as sns #i have installed seaborn in my terminal to make it work, because otherwise I got an
error message that the library is not found
#I used this website to download it: https://seaborn.pydata.org/installing.html
sns.kdeplot(x='nr.employed', data=bank_data)
plt.title('Distribution of emplyoment')
plt.xlabel('number of people in job')
plt.ylabel('')
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
# two categorical variable plot
# first choice is job
sns.countplot(x='job', data=bank_data, palette='Set2')
plt.title('people in jobs distribution')
plt.xlabel('job types')
plt.ylabel('Count')
plt.show()
```

#I used this type of plot, because it is the best way to show the distribution of categorical variables, and it is also easy to read and understand

/var/folders/zy/lbmjr89s4k50gwpxfwb5pzv40000gn/T/ipykernel_19896/3060217927.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='job', data=bank_data, palette='Set2')
```

<Figure size 640x480 with 1 Axes>

```
#second choice of categorical variable
sns.countplot(x='marital', data=bank_data, palette='Set2')
plt.title('people in relationships distribution')
plt.xlabel('marital status')
plt.ylabel('Count')
plt.show()
```

/var/folders/zy/lbmjr89s4k50gwpxfwb5pzv40000gn/T/ipykernel_19896/122631489.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='marital', data=bank_data, palette='Set2')
```

<Figure size 640x480 with 1 Axes>

Task 4 Managing Missing Values

Lecture material: Lecture 2 (Data Inspection), Lecture 5 (Preprocessing and Pipeline Discipline).

- Identify both explicit missing values (e.g., NaN) and implicit missing values (e.g., categories such as unknown or sentinel numerical values, i.e., values that may represent special codes rather than genuine measurements).

- Quantify the extent of missingness for each affected variable.
- Propose and justify a strategy for handling missing values in each case (e.g., removal, imputation, separate category, indicator variable).
- Clearly state which operations must be fitted using the training set only, and explain why. ???

Note: Your strategy should distinguish between “data cleaning” decisions (e.g., correcting inconsistent entries) and “modeling” decisions (e.g., whether missingness itself may carry predictive information).

My answer

- * explicit missing value
- * poutcome
- * implicit missing value
- * default
- * housing
- * loan
- * quantifying value missingness is in my code document
- * My strategy for handling missing values is keeping them in the data set, but marking them with a different value. As it is visible in my code, I marked them with NA since it is a unique value for the missing values. The reason I would not remove them is because the dataset is not large enough in my judgement so I would rather not remove values which would desort the outcome. So in my best judgement, missing values may carry predictive information
- *

AI prompt disclaimer

- * Question asked: How do i modify the table to put null for missing values so the .isnull works?
- * Answer received:
- * df.head()

df['job'].unique()

```
df = pd.read_csv('bank-additional-full.csv', sep=';')
#in the following code lines I inspect how is the missing value encoded in the dataset
#i only keep those which have missing values
df.head()
df['job'].unique() #this piece of code shows all the unique types of jobs which appear in the job column
```

```
array(['housemaid', 'services', 'admin.', 'blue-collar', 'technician',
       'retired', 'management', 'unemployed', 'self-employed', 'unknown',
       'entrepreneur', 'student'], dtype=object)
```

```
df.head()
df['marital'].unique()
#this piece of code shows all the unique types of marital status which appear in the marital column
```

```
array(['married', 'single', 'divorced', 'unknown'], dtype=object)
```

```
df.head()
df['education'].unique()
#this piece of code shows all the unique types of education which appear in the education column
```

```
array(['basic.4y', 'high.school', 'basic.6y', 'basic.9y',  
      'professional.course', 'unknown', 'university.degree',  
      'illiterate'], dtype=object)
```

```
df.head()  
df['default'].unique()  
#this piece of code shows all the unique types of default which appear in the default column  
  
array(['no', 'unknown', 'yes'], dtype=object)
```

```
df.head()  
df['housing'].unique()  
#this piece of code shows all the unique types of housing which appear in the housing column  
  
array(['no', 'yes', 'unknown'], dtype=object)
```

```
df.head()  
df['loan'].unique()  
#this piece of code shows all the unique types of loan which appear in the loan column  
  
array(['no', 'yes', 'unknown'], dtype=object)
```

```
df.head()  
df['poutcome'].unique()  
#this piece of code shows all the unique types of poutcome which appear in the poutcome column  
  
array(['nonexistent', 'failure', 'success'], dtype=object)
```

#after inspecting the unique values of the columns, I can see that the missing value is encoded as 'unknown' in the dataset, so I will replace it with NA for better handling of missing values in pandas

```
cols = ['job', 'marital', 'education', 'default', 'housing', 'loan']
```

```
df[cols] = df[cols].replace('unknown', pd.NA)
```

#in poutcome column, I also replaced the 'nonexistent' value with NA

```
cols = ['poutcome']  
#I had to do this separately, because the missing value is encoded differently in this column, so I had to replace it with NA separately  
df[cols] = df[cols].replace('nonexistent', pd.NA)
```

#after replacing the missing value with NA, I will check how many missing values are in the dataset and what percentage of the dataset is missing
#and also verify that all the replacing process has worked properly
#so here I Quantify the extent of missingness for each affected variable.

```
missing_summary = pd.DataFrame({  
    "Missing Count": df.isnull().sum(),  
    "Missing %": df.isnull().mean() * 100  
}).round(2)
```

missing_summary

	Missing Count	Missing %
age	0	0.00
job	330	0.80
marital	80	0.19
education	1731	4.20
default	8597	20.87
housing	990	2.40
loan	990	2.40
contact	0	0.00
month	0	0.00
day_of_week	0	0.00
duration	0	0.00
campaign	0	0.00
pdays	0	0.00
previous	0	0.00
poutcome	35563	86.34
emp.var.rate	0	0.00

cons.price.idx	0	0.00
cons.conf.idx	0	0.00
euribor3m	0	0.00
nr.employed	0	0.00
y	0	0.00

```
sns.kdeplot(x='Missing Count', data=missing_summary)
#I just plotted this out of pure interest of mine
plt.show()
```

<Figure size 640x480 with 1 Axes>

From this table we can see that there is missing data in the following columns: job, marital, education, default, housing, loan, poutcome. So I will have to pay attention to these columns.

Task 5: Encoding Categorical Variables

Lecture material: Lecture 4 (Categorical Encoding), Lecture 6 (Linear Models), Lecture 9 (Feature Engineering and Expressiveness).

- Identify all categorical variables in the dataset.
- Distinguish between nominal variables (categories without intrinsic order, e.g., job type) and ordinal variables (categories with a meaningful order, e.g., education level), and justify your classification.
- Select and apply an appropriate encoding strategy for each categorical variable. -> applied in my code pdf and jupyter notebook
- Clearly state which encoders must be fitted on the training set only, and explain why.
- Analyze how encoding changes:
 - the dimensionality of the dataset;
 - the interpretability of model coefficients;
 - the types of decision boundaries a linear model can represent.

Note: Encoding is not a purely mechanical transformation. Your justification should explicitly connect your encoding decisions to the assumptions and behavior of Logistic Regression

- * My answer
- * categorical variables
- * education
- * contact
- * job
- * marital
- * poutcome

* Nominal variables

* contact

* job

* marital

* poutcome

* Ordinal variables

* education

* My reason behind this classification is that ordinal variables are variables that can be put in some ascending or descending order. In this case education can be put into order for example primary school -> high school -> Bachelor's degree -> Master's degree and so on. However the categorical variables cannot be put into such order. They are all on the "same level" and we cannot construct an order from those variables. So there is no meaningful order.

* Chosen encoding method -> I will use one hot encoding. See the application in my code.

* In my opinion and according to my knowledge those encoders should be fitted to the training set only which have none or very few (1-2) missing values from the encoded categorical variables. It is because the model should have accessible data which it can learn from and no out-of-range variables which only add to the computational time.

* the dimensionality of the dataset;

* the interpretability of model coefficients;

* the types of decision boundaries a linear model can represent.

```
print(df.columns.tolist()) #printing all the columns from the dataset
```

```
['age', 'job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'day_of_week',  
'duration', 'campaign', 'pdays', 'previous', 'poutcome', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx',  
'euribor3m', 'nr.employed', 'y']
```

```
df = pd.read_csv('bank-additional-full.csv', sep=';')
```

```
cat_cols = ['education', 'contact', 'job', 'marital', 'poutcome']
```

```
#these are the categorical columns which I will one hot encode
```

```
df_oh = pd.get_dummies(df, columns=cat_cols, dtype=int)
```

```
#here the categorical variables get 1 and 0 values, depending on existence or missingness
```

```
#printing out encoded columns to see and be able to inspect them if necessary
```

```
encoded_cols = [col for col in df_oh.columns if any(col.startswith(c + '_') for c in cat_cols)]
```

```
print("\n=== AFTER encoding ===")
```

```
display(df_oh[encoded_cols].head(10))
```

```
=== AFTER encoding ===
```

	education_basic.4y	education_basic.6y	education_basic.9y	\
0	1	0	0	
1	0	0	0	
2	0	0	0	
3	0	1	0	
4	0	0	0	
5	0	0	1	
6	0	0	0	
7	0	0	0	
8	0	0	0	
9	0	0	0	

	education_high.school	education_illiterate	education_professional.course	\
0	0	0	0	
1	1	0	0	
2	1	0	0	
3	0	0	0	

4	1	0	0
5	0	0	0
6	0	0	1
7	0	0	0
8	0	0	1
9	1	0	0

	education_university.degree	education_unknown	contact_cellular	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0			

Task 6: Feature Selection

Lecture material: Lecture 5 (Feature Selection), Lecture 6 (Linear Models), Lecture 9 (Pipeline Discipline).

- Identify and remove features with very low variance, if any. Justify the criterion used to define “low” variance.
- Identify highly correlated numerical features and decide whether any should be removed. Clearly state the threshold used and justify your decision.
- Discuss whether any features should be removed based on conceptual considerations (e.g., redundancy, availability at prediction time, or risk of data leakage).
- Explain why feature selection must be performed using the training set only.
- Discuss the consequences of performing feature selection on the entire dataset before splitting.

Note: Feature selection is not purely statistical. Your reasoning should explicitly connect your decisions to the assumptions and stability of Logistic Regression

My answers

* The 'education_illiterate', 'job_unknown', 'marital_unknown' has been removed due to very low variance. Also, rare categories add noise and rarely help the prediction of the target variable. So the reason why I determined it as a "low variance" feature is because it is significantly less than other variables as seen in my code.

* After doing some research, about 0.8 is the threshold used for correlation. This seems like a valid threshold since anything correlated above this may affect the algorithm negatively. I would most probably remove the features which are shown with dark red or dark blue on the correlation matrix that has been visualised with the Python code.

* I believe the features we should be attention to is the 'default', 'previous', 'poutcome' feature because these maybe available at prediction time and cause data leakage. Those have been removed from the data set due to low variance.

* Feature selection should be performed using the training set only because the model will base its predictions and "knowledge" on the training data, which will highly determine what answer we have from the model. (Hopefully not just 42 :D) So we should make sure that it only contains the features that are needed for us. Also, this prevents data leakage and bias in the model. We can also check for independence within the training data and check if there is not large outliers.

* Performing feature selection on the entire dataset before splitting is not a great solution. The model will not recognize if there is any unusual when we provide the test and validation set since we have done feature selection on the entire dataset and have most probably discarded the unwanted features. So technically the model will be useless in short.

```
# Check variance of all columns in the encoded dataframe
variance = df_oh.var(numeric_only=True).sort_values()
print(variance)
```

education_illiterate	0.000437
marital_unknown	0.001939
job_unknown	0.007948
job_student	0.020793
job_unemployed	0.024013
job_housemaid	0.025074
poutcome_success	0.032225
job_self-employed	0.033311
job_entrepreneur	0.034101
job_retired	0.040017
education_unknown	0.040262
education_basic.6y	0.052552
job_management	0.065953
job_services	0.087079
education_basic.4y	0.091111
poutcome_failure	0.092579
marital_divorced	0.099439
education_professional.course	0.111093
poutcome_nonexistent	0.117921
education_basic.9y	0.125229
job_technician	0.136914
job_blue-collar	0.174202
education_high.school	0.177651
job_admin.	0.189013
marital_single	0.201982
education_university.degree	0.208154
contact_telephone	0.231849
contact_cellular	0.231849
marital_married	0.238934
previous	0.244927
cons.price.idx	0.335056
emp.var.rate	2.467915
euribor3m	3.008308
campaign	7.672975
cons.conf.idx	21.420215
age	108.602451
nr.employed	5220.283250
pdays	34935.687284
duration	67225.728877

dtype: float64

```
from sklearn.feature_selection import VarianceThreshold

# Encode the target column 'y' first
df_oh['y'] = df_oh['y'].map({'yes': 1, 'no': 0})

# Drop any remaining non-numeric columns before fitting
X = df_oh.drop(columns=['y']).select_dtypes(include='number')

selector = VarianceThreshold(threshold=0.01)
selector.fit(X)

# See which features are removed later in printing
all_cols = X.columns
removed_cols = all_cols[~selector.get_support()]
kept_cols = all_cols[selector.get_support()]

print(f"Features removed ({len(removed_cols)}): {removed_cols.tolist()}")
print(f"Features kept: {len(kept_cols)}")
```

```
Features removed (3): ['education_illiterate', 'job_unknown', 'marital_unknown']
Features kept: 36
```

```
df_reduced = df_oh[kept_cols.tolist() + ['y']]
#here the original and reduced shape are shown, and also the number of features which were dropped
print(f"Original shape: {df_oh.shape}")
print(f"Reduced shape: {df_reduced.shape}")
print(f"Features dropped: {df_oh.shape[1] - df_reduced.shape[1]}")
```

```
Original shape: (41188, 45)
Reduced shape: (41188, 37)
```

Features dropped: 8

```
#testing high coreltaion between the features
# Select only numeric columns
numeric_df = df_reduced.select_dtypes(include='number').drop(columns=['y'])

# Compute correlation matrix
corr_matrix = numeric_df.corr()
print(corr_matrix)
```

	age	duration	campaign	pdays	\
age	1.000000	-0.000866	0.004594	-0.034369	
duration	-0.000866	1.000000	-0.071699	-0.047577	
campaign	0.004594	-0.071699	1.000000	0.052584	
pdays	-0.034369	-0.047577	0.052584	1.000000	
previous	0.024365	0.020640	-0.079141	-0.587514	
emp.var.rate	-0.000371	-0.027968	0.150754	0.271004	
cons.price.idx	0.000857	0.005312	0.127836	0.078889	
cons.conf.idx	0.129372	-0.008173	-0.013733	-0.091342	
euribor3m	0.010767	-0.032897	0.135133	0.296899	
nr.employed	-0.017725	-0.044703	0.144095	0.372605	
education_basic.4y	0.244079	0.008476	0.004000	0.005804	
education_basic.6y	0.009897	0.005739	-0.000991	0.023238	
education_basic.9y	-0.038320	0.004810	-0.005278	0.036963	
education_high.school	-0.106550	0.005500	0.000195	0.005522	
education_professional.course	0.002054	-0.008472	0.002554	-0.003493	
education_university.degree	-0.071138	-0.012641	-0.000950	-0.036958	
education_unknown	0.069485	0.003317	0.002162	-0.022014	
contact_cellular	-0.007021	0.026657	-0.077368	-0.117970	
contact_telephone	0.007021	-0.026657	0.077368	0.117970	
job_admin.	-0.102584	-0.008918	0.011745	-0.025398	
job_blue-collar	-0.024191	0.012992	-0.001775	0.065335	
job_entrepreneur	0.031213	0.003679	-0.002203	0.019246	
job_housemaid	0.085403	-0.004908	0.004226	-0.001649	
job_management	0.062040	-0.001308	-0.009135	0.000254	
job_retired	0.440772	0.012421	-0.006847	-0.072084	
job_self-employed	-0.001356	0.004270	0.006361	0.014307	
job_services					

```
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
```

```
# Set threshold
threshold = 0.8
```

```
# Find feature pairs above threshold, to be able to inspect them and decide if I want to drop any of them,
because they are highly correlated
```

```
#also to be able to nicely visualise them on the next step
```

```
correlated_pairs = [
    (col, row, upper.loc[row, col])
    for col in upper.columns
    for row in upper.index
    if abs(upper.loc[row, col]) > threshold
]
```

```
pairs_df = pd.DataFrame(correlated_pairs, columns=['Feature 1', 'Feature 2', 'Correlation'])
pairs_df = pairs_df.sort_values('Correlation', ascending=False)
display(pairs_df)
```

	Feature 1	Feature 2	Correlation
0	euribor3m	emp.var.rate	0.972245
2	nr.employed	euribor3m	0.945154
1	nr.employed	emp.var.rate	0.906970
5	poutcome_nonexistent	poutcome_failure	-0.853119
4	poutcome_nonexistent	previous	-0.878776
6	poutcome_success	pdays	-0.950700
3	contact_telephone	contact_cellular	-1.000000

```
# Visualize the correlation matrix
plt.figure(figsize=(14, 10))
sns.heatmap(
    corr_matrix,
```



```

    annot=False,
    cmap='coolwarm',
    center=0,
    vmin=-1, vmax=1,
    linewidths=0.5
)
plt.title('Feature Correlation Matrix')
plt.tight_layout()
plt.show()

```

<Figure size 1400x1000 with 2 Axes>

Task 7: Data Splitting

Lecture material: Lecture 2 (Data Splitting and Leakage), Lecture 9 (ML Pipeline).

- Split the dataset into training, validation, and test sets.
- Justify your choice of proportions for each split.
- Perform stratified splitting with respect to the target variable and explain why stratification is necessary for this dataset.
- Clearly describe at which stage of your pipeline the split must occur, and explain what types of data leakage would arise if splitting were performed later.

Note: A recommended strategy is to first split the dataset into a training set and a temporary set, and then split the temporary set into validation and test sets. Use the stratify argument of train test split where appropriate

My answer:

* The reason I chose the 80-20 split is because it is one of the common ones and in my opinion 80% of data is enough to train the model and the remaining 20% is the validation + test set.

* I have already done stratified splitting and stratification is highly recommended to maintain the class balances. Also to ensure that data is accurately represented especially when they are imbalanced.

* In my pipeline the splitting occurs after feature selection, encoding and missing value management because first I wanted to make sure that the data is ready to be split. Readiness means that there is an off range value set for missing values, the categorical values are properly encoded and there is a feature that can give a guide how to go further with my work. If I have performed the splitting later I believe target data leakage would have occurred. This type of leakage is when information about the target variable is leaked for the model in some point of the training.

```

from sklearn.model_selection import train_test_split
from sklearn.feature_selection import VarianceThreshold
import pandas as pd

#https://www.geeksforgeeks.org/machine-learning/how-to-split-a-dataset-into-train-and-test-sets-using-python/
df_fresh = pd.read_csv('bank-additional-full.csv', sep=';')

cat_cols = ['education', 'contact', 'job', 'marital', 'outcome']
df_fresh = pd.get_dummies(df_fresh, columns=cat_cols, dtype=int)

# Encode target
df_fresh['y'] = df_fresh['y'].map({'yes': 1, 'no': 0})

#apply variance threshold

```

```

X_all = df_fresh.drop(columns=['y']).select_dtypes(include='number')
selector = VarianceThreshold(threshold=0.01)
selector.fit(X_all)
kept_cols = X_all.columns[selector.get_support()]

X = df_fresh[kept_cols] #the features are all the columns except the target column 'y'
y = df_fresh['y'] #the target variable is the column 'y' which is the subscription status

# Confirm no NaN before splitting
print("NaN in y:", y.isna().sum())
print(y.value_counts(dropna=False))

# 80-20 split with stratification
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2, #the 80-20 split is the most common to split data, so I also use these values for my
    split
    random_state=42,
    stratify=y
)

print(X_train.shape, X_test.shape)
print(y_train.value_counts(normalize=True), "\n", y_test.value_counts(normalize=True))

```

```

NaN in y: 0
y
0    36548
1     4640
Name: count, dtype: int64
(32950, 36) (8238, 36)
y
0    0.887344
1    0.112656
Name: proportion, dtype: float64
y
0    0.887351
1    0.112649
Name: proportion, dtype: float64

```

```

X_val, X_test, y_val, y_test = train_test_split(
    X_test, y_test,
    test_size=0.5,          #10% test, 10% validation
    random_state=42,
    stratify=y_test
)

print("Train:", X_train.shape)
print("Validation:", X_val.shape)
print("Test:", X_test.shape)

```

```

Train: (32950, 36)
Validation: (4119, 36)
Test: (4119, 36)

```

Task 8 Addressing Class Imbalance

Lecture material: Lecture 3 (Class Imbalance), Lecture 4 (Evaluation Metrics), Lecture 9 (Pipeline Discipline).

- Quantify the class distribution in the training set and explain why imbalance is or is not a concern for this prediction task.
- Propose and apply a resampling strategy (e.g., random oversampling, SMOTE, or ADASYN). Clearly justify at which stage of the pipeline the resampling step should occur.

- Justify your choice of resampling method in terms of its assumptions and expected effect on the learning algorithm.
- Explain what would happen if resampling were applied before splitting the dataset into training, validation, and test sets. Discuss the implications for model evaluation.
- Briefly discuss how class imbalance affects evaluation metrics such as accuracy, precision, and recall.

Note: Resampling is part of the training procedure and must be applied to the training set only. Validation and test sets must preserve the original class distribution

My answer

* The class imbalance is a bit of a conert for this prediction task because the prediction model will mostly learn instances where a customer did not sign the contract offered by the company. So from this the model will learn that with similar features the customer will not sign up, so the model learns as such. So, this imbalance has to be balanced out to make sure the model is not biased towards the features of a non signing customer

* I will be using random oversampling as a resampling strategy. I believe it should occur after data splitting because as far as I know random oversampling randomly duplicates instances of the minority class which may disort the data for the model which we cannot control since it is random. Therefore we cannot control what does the model learn from the newly balanced data.

* I would like to justify my choice of resampling with a few of its assumptions and expected effect on the leaerning algorithm

* One of the assumptions is that the observations are independent. In the case of the banking data, whether customers sign up or not is independet from eachother. Since it is assumed that the customers do not influence eachother into choosing one or another. As of expected effects, it does what it should, meaning it with random choices it creates class imbalance and in my opinion this may cause the learning algorithm to learn features duplicated, due to the random sampling of the minority class.

* What would happen before splitting the dataset is that information from the test set can leak into the training set due to overlapping resampled data. Which would cause a data leakage through preprocessing if we are not careful. The implications of model evaluations would be that the model would be biased in prediction and would not predict accurately those customers that slitely differ from those in the test data. So in short the model would be a bit useless.

* As we can see from the confusion matrix:

* precision very much changes after oversampling

* recall only changes slightly

* and accuracy from my calculations only changes slightly

```
...
AI prompt: "How to quantify the class distribution in the training set in python"
I have asked this question to AI because I am not sure how to do it in python, I did not give it the
dataset, that is a thing I edited in myself.

Answer I receivvd from AI:

...
# Count and percentage of each class in the training set
print("Class counts:")
print(y_train.value_counts())

print("\nClass proportions:")
print(y_train.value_counts(normalize=True))

...
So, as we see from the output, there are 29238 instances of class 0 (not subscribed) and 3712 instances of
class 1 (subscribed) in the training set.
Which are as class proportions, 88.73% of the training set belongs to class 0 and 11.26% belongs to class
1.
This indicates that the dataset is imbalanced, with a much higher proportion of non-subscribed instances
```

```
compared to subscribed instances.
...
```

```
Class counts:
```

```
y
0    29238
1     3712
Name: count, dtype: int64
```

```
Class proportions:
```

```
y
0    0.887344
1    0.112656
Name: proportion, dtype: float64
```

```
'\nSo, as we see from the output, there are 29238 instances of class 0 (not subscribed) and 3712 instances of class 1 (subscribed) in the training set.\nWhich are as class proportions, 88.73% of the training set belongs to class 0 and 11.26% belongs to class 1. \nThis indicates that the dataset is imbalanced, with a much higher proportion of non-subscribed instances compared to subscribed instances.\n'
```

```
#I have installed "sudo pip install imbalanced-learn" to do random oversampling
# check version number
import imblearn
print(imblearn.__version__)
```

```
0.14.1
```

```
df = pd.read_csv('bank-additional-full.csv', sep=';')
df.head()
#i am once again loading the original dataset
```

```
   age  job  marital  education  default  housing  loan  contact  \
0   56  housemaid  married   basic.4y     no      no   no  telephone
1   57  services  married  high.school  unknown     no   no  telephone
2   37  services  married  high.school     no     yes   no  telephone
3   40   admin.  married   basic.6y     no      no   no  telephone
4   56  services  married  high.school     no      no   yes  telephone

   month  day_of_week  ...  campaign  pdays  previous  poutcome  emp.var.rate  \
0   may           mon  ...         1     999         0  nonexistent         1.1
1   may           mon  ...         1     999         0  nonexistent         1.1
2   may           mon  ...         1     999         0  nonexistent         1.1
3   may           mon  ...         1     999         0  nonexistent         1.1
4   may           mon  ...         1     999         0  nonexistent         1.1

   cons.price.idx  cons.conf.idx  euribor3m  nr.employed  y
0         93.994         -36.4      4.857      5191.0  no
1         93.994         -36.4      4.857      5191.0  no
2         93.994         -36.4      4.857      5191.0  no
3         93.994         -36.4      4.857      5191.0  no
4         93.994         -36.4      4.857      5191.0  no
```

```
[5 rows x 21 columns]
```

```
#Split our data into features(X) and labels(y)
X=df.drop('y',axis=1)
y=df['y']
y.value_counts()
#this is the counts before oversampling, we can see that there are 36548 instances of class no and 4640 instances of class yes, which means that the dataset is imbalanced, with a much higher proportion of non-subscribed instances compared to subscribed instances.
```

```
y
no    36548
yes    4640
Name: count, dtype: int64
```

```
y.value_counts().plot.pie(autopct='%.2f')
#just to visualize the distribution of the target variable before oversampling
```

```
<Axes: ylabel='count'>
```

```
<Figure size 640x480 with 1 Axes>
```

```
from imblearn.over_sampling import RandomOverSampler
ros = RandomOverSampler(sampling_strategy="not majority")
X_res, y_res = ros.fit_resample(X, y)
ax = y_res.value_counts().plot.pie(autopct='%.2f')
_ = ax.set_title("Over-sampling")
```

```
<Figure size 640x480 with 1 Axes>
```

```
y_res.value_counts()
#here we can see that after oversampling, the class distribution is now balanced, with 36548 instances of
class no and 36548 instances of class yes, which means that both classes now represent 50% of the dataset.
```

```
y
no    36548
yes   36548
Name: count, dtype: int64
```

AI disclaimer:

- * I have asked a generative AI on how to create a confusion matrix in python
- * And I have received a code similar to this
- * However the answer to the point in the task description is answered by myself for a 100%

```
#confusion matrix before oversampling
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report
import matplotlib.pyplot as plt

# Encode target for the train/val split (from df_reduced) from task 7
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit on train only
X_val_scaled = scaler.transform(X_val) # transform val with same scaler

# Train on original imbalanced training data
lr_imbalanced = LogisticRegression(max_iter=1000, random_state=42)
lr_imbalanced.fit(X_train_scaled, y_train)

# Predict on validation set
y_pred_imbalanced = lr_imbalanced.predict(X_val_scaled)

# Plot confusion matrix
cm = confusion_matrix(y_val, y_pred_imbalanced)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No (0)', 'Yes (1)'])
disp.plot(cmap='Blues')
plt.title('Confusion Matrix - Before Oversampling (Imbalanced)')
plt.tight_layout()
plt.show()

# Print metrics
print(classification_report(y_val, y_pred_imbalanced, target_names=['No', 'Yes']))
```

```
<Figure size 640x480 with 2 Axes>
```

	precision	recall	f1-score	support
No	0.92	0.98	0.95	1827
Yes	0.67	0.35	0.46	232
accuracy			0.91	2059
macro avg	0.79	0.67	0.71	2059
weighted avg	0.89	0.91	0.89	2059

```
#confusion matrix after oversampling
from imblearn.over_sampling import RandomOverSampler

# Apply oversampling to training set ONLY
ros = RandomOverSampler(random_state=42)
X_train_res, y_train_res = ros.fit_resample(X_train_scaled, y_train)

# Train on balanced training data
lr_balanced = LogisticRegression(max_iter=1000, random_state=42)
lr_balanced.fit(X_train_res, y_train_res)

# Predict on the validation set -> no resampling here
y_pred_balanced = lr_balanced.predict(X_val_scaled)

# Plot confusion matrix
cm2 = confusion_matrix(y_val, y_pred_balanced)
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm2, display_labels=['No (0)', 'Yes (1)'])
disp2.plot(cmap='Greens')
plt.title('Confusion Matrix – After Oversampling (Balanced)')
plt.tight_layout()
plt.show()

# Print metrics
print(classification_report(y_val, y_pred_balanced, target_names=['No', 'Yes']))
```

<Figure size 640x480 with 2 Axes>

	precision	recall	f1-score	support
No	0.99	0.87	0.92	1827
Yes	0.46	0.90	0.61	232
accuracy			0.87	2059
macro avg	0.72	0.88	0.77	2059
weighted avg	0.93	0.87	0.89	2059

Task 9: Feature Scaling

Lecture material: Lecture 5 (Feature Scaling), Lecture 6 (Logistic Regression and Optimization)..

- Identify the numerical variables that require scaling.
- Select and apply an appropriate scaling strategy (e.g., standardization or normalization) to those variables.
- Justify your choice of scaling method in the context of Logistic Regression.
- Clearly state which transformations must be fitted on the training set only, and explain why.
- Discuss how feature scaling affects
 - gradient-based optimization;
 - the magnitude and comparability of model coefficients;
 - the interpretation of regularization penalties.

Note: Feature scaling is not a cosmetic transformation. Your justification should explicitly connect your scaling decision to

the mathematical behavior of linear models

my answer:

* I am not really sure what transformation means, but I believe all of these which I included in this section, can only be fitted onto the training data because the validation and test set has to keep their original distribution.

* Affect of feature scaling:

* My feature scaling affects the gradient-based optimisation with the min-max scaling. It happens by alternating the different variables that I did the scaling with. For example different with age or duration, just to say a few examples.

* The way scaling affects the magnitude and comparability of model coefficients my scaling is that as I change the independent variables by scaling then there is a change in the dependent variable too.

* Finally, regularisation penalties are affected such a way that the hyperparameters are a subject to change before training and that determines the penalty for regularisation

First I will be checking which variable needs scaling

```
bank_data.describe() # Look at original min, max, mean, std for each column
#If min and max values differ significantly, I may need to scale
#If there is large differences in standard deviation between features they may need to be scaled before
training my model
```

	age	duration	campaign	pdays	previous \
count	41188.00000	41188.00000	41188.00000	41188.00000	41188.00000
mean	40.02406	258.28501	2.567593	962.475454	0.172963
std	10.42125	259.279249	2.770014	186.910907	0.494901
min	17.00000	0.00000	1.00000	0.00000	0.00000
25%	32.00000	102.00000	1.00000	999.00000	0.00000
50%	38.00000	180.00000	2.00000	999.00000	0.00000
75%	47.00000	319.00000	3.00000	999.00000	0.00000
max	98.00000	4918.00000	56.00000	999.00000	7.00000

	emp.var.rate	cons.price.idx	cons.conf.idx	euribor3m	nr.employed
count	41188.00000	41188.00000	41188.00000	41188.00000	41188.00000
mean	0.081886	93.575664	-40.502600	3.621291	5167.035911
std	1.570960	0.578840	4.628198	1.734447	72.251528
min	-3.400000	92.201000	-50.800000	0.634000	4963.600000
25%	-1.800000	93.075000	-42.700000	1.344000	5099.100000
50%	1.100000	93.749000	-41.800000	4.857000	5191.000000
75%	1.400000	93.994000	-36.400000	4.961000	5228.100000
max	1.400000	94.767000	-26.900000	5.045000	5228.100000

```
bank_data['age'].plot(kind='box', subplots=True, figsize=(12, 6))
plt.show()
#plotting age for visualization
```

<Figure size 1200x600 with 1 Axes>

```
bank_data['duration'].plot(kind='box', subplots=True, figsize=(12, 6))
plt.show()
#visualisation of call duration
```

<Figure size 1200x600 with 1 Axes>

```
bank_data['campaign'].plot(kind='box', subplots=True, figsize=(12, 6))
plt.show()
#visualisation of campaign
```

<Figure size 1200x600 with 1 Axes>

```
bank_data['pdays'].plot(kind='box', subplots=True, figsize=(12, 6))
plt.show()
#visualisation of pdays
```

<Figure size 1200x600 with 1 Axes>

```
bank_data['previous'].plot(kind='box', subplots=True, figsize=(12, 6))
plt.show()
#visualisation of previous
```

<Figure size 1200x600 with 1 Axes>

```
bank_data['emp.var.rate'].plot(kind='box', subplots=True, figsize=(12, 6))
plt.show()
#visualisation of emp.var.rate
```

<Figure size 1200x600 with 1 Axes>

```
bank_data['cons.conf.idx'].plot(kind='box', subplots=True, figsize=(12, 6))
plt.show()
# visualisation of cons.conf.idx
```

<Figure size 1200x600 with 1 Axes>

```
bank_data['euribor3m'].plot(kind='box', subplots=True, figsize=(12, 6))
plt.show()
# visualisation of euribor3m
```

<Figure size 1200x600 with 1 Axes>

```
bank_data['nr.employed'].plot(kind='box', subplots=True, figsize=(12, 6))
plt.show()
# visualisation of nr.employed
```

<Figure size 1200x600 with 1 Axes>

Scaling

I have decided to use min-max scaling and normalisation so I can make a comparison on the usage of either.

The features I have decided to scale, due to the large differences in min-max and variance.

These large differences are also a good reason why I will be using min-max and normalisation scaling, so hopefully they will even out.

* age

* duration

* campaign

* pdays

* previous

I used my code from this website and ofcourse modified them as I needed.

<https://www.analyticsvidhya.com/blog/2021/05/feature-scaling-techniques-in-python-a-complete-guide/#h-normalization>

I have used these scaling methods in the context of logistic regression is because I am assuming independent observations per client of this banking data and also because there is a pretty large sample size to work with in this dataset. Finally, because the dependent variable which we are trying to predict, takes only binary values.

```
meanage = bank_data['age'].mean()
meanduration = bank_data['duration'].mean()
meancampaign = bank_data['campaign'].mean()
meanpdays = bank_data['pdays'].mean()
meanprevious = bank_data['previous'].mean()

#with these codes I only calculate the mean of the numerical features
#I will use these values for normalisation in the next step
```

```
minage = bank_data['age'].min()
minduration = bank_data['duration'].min()
mincampaign = bank_data['campaign'].min()
minpdays = bank_data['pdays'].min()
minprevious = bank_data['previous'].min()

#here I am doing the same for minimum
```

```
maxage = bank_data['age'].max()
maxduration = bank_data['duration'].max()
maxcampaign = bank_data['campaign'].max()
maxpdays = bank_data['pdays'].max()
maxprevious = bank_data['previous'].max()

#here I am doing the same for maximum
```

```
ages = bank_data['age']
duration = bank_data['duration']
campaign = bank_data['campaign']
pdays = bank_data['pdays']
previous = bank_data['previous']

#getting each value one by one for normalisation
```

```
#I am starting with normalisation and plugging in the values of mean, minimum, maximum for each feature
# the formula is from the website I put to this task
age = (ages-meanage)/(maxage-minage)
duration = (duration-meanduration)/(maxduration-minduration)
campaign = (campaign-mincampaign)/(maxcampaign-mincampaign)
pdays = (pdays-minpdays)/(maxpdays-minpdays)
previous = (previous-minprevious)/(maxprevious-minprevious)
```

```
#here I visualise the normalised features, to see how they look like after normalisation, and to check if
there are any outliers or if the distribution is more balanced after normalisation
x = range(len(df))
plt.plot(x, age, 'red')
plt.title('Normalised Numerical Features')
plt.xlabel('Index')
plt.ylabel('Normalised Value')
plt.legend(['age', 'duration', 'campaign', 'pdays', 'previous'])
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
x = range(len(df))
plt.plot(x, duration, 'blue')
plt.title('Normalised Numerical Features')
plt.xlabel('Index')
plt.ylabel('Normalised Value')
```

```
plt.legend(['age', 'duration', 'campaign', 'pdays', 'previous'])
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
x = range(len(bank_data))
plt.plot(x,campaign,'green')
plt.title('Normalised Numerical Features')
plt.xlabel('Index')
plt.ylabel('Normalised Value')
plt.legend(['age', 'duration', 'campaign', 'pdays', 'previous'])
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
x = range(len(bank_data))
plt.plot(x,pdays,'orange')
plt.title('Normalised Numerical Features')
plt.xlabel('Index')
plt.ylabel('Normalised Value')
plt.legend(['age', 'duration', 'campaign', 'pdays', 'previous'])
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
x = range(len(bank_data))
plt.plot(x,previous,'purple')
plt.title('Normalised Numerical Features')
plt.xlabel('Index')
plt.ylabel('Normalised Value')
plt.legend(['age', 'duration', 'campaign', 'pdays', 'previous'])
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
#now comes the min-max scaling
a = (ages-minage)/(maxage-minage)
d = (duration-minduration)/(maxduration-minduration)
c = (campaign-mincampaign)/(maxcampaign-mincampaign)
pd = (pdays-minpdays)/(maxpdays-minpdays)
p = (previous-minprevious)/(maxprevious-minprevious)
```

```
#here I am visualising after min-max scaling, to see how the features look like after min-max scaling, and
to check if there are any outliers or if the distribution is more balanced after min-max scaling
x = range(len(df))
plt.plot(x, a,'red')
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
plt.plot(x, d,'blue')
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
plt.plot(x, c,'green')
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
plt.plot(x, pd,'orange')
plt.show()
```

<Figure size 640x480 with 1 Axes>

```
plt.plot(x, p,'purple')
plt.show()
```

Task 10 training a Logistic Regression Model

Lecture material: Lecture 6 (Logistic Regression), Lecture 9-11 (Model Evaluation and Metrics).

- Train a Logistic Regression model to predict whether a client subscribes to a term deposit.
- Use the validation set to generate predictions.
- Report at least Accuracy, Precision, and Recall on the validation set.
- Compare the model's accuracy with the Zero Rule baseline and briefly interpret the result.

Note: The goal here is not to squeeze out the best possible performance. The goal is to verify that your data preparation pipeline is coherent and correctly implemented. If your preprocessing is principled, the model should behave sensibly. If it behaves strangely, that is a signal to revisit earlier decisions. Have fun finding a visually appealing way to display the predictions or the confusion matrix on the validation set. This is your chance to make the output readable and professional 8-)

my review:

- * This model seems alright and working, hopefully everything is right what I did so far and the way I explained it.
- * After grading I would be happy to review it if there are any missing or wrong evaluation I did.

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

# 1. Preprocess the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 2. train the model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# 3. model accuracy evaluation
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.911143481427531
              precision    recall  f1-score   support

    0           0.93       0.98        0.95       3655
    1           0.69       0.39        0.49        464

   accuracy                   0.91       4119
  macro avg           0.81       0.68        0.72       4119
 weighted avg           0.90       0.91        0.90       4119
```

```
#generation predictions on the validation set
y_val_pred = model.predict(X_val) # Predicted class labels
y_val_pred_proba = model.predict_proba(X_val) # Predicted probabilities per class

print("Validation Accuracy:", accuracy_score(y_val, y_val_pred))
print(classification_report(y_val, y_val_pred))
print("Predicted probabilities (first 5):\n", y_val_pred_proba[:5])
```

```
Validation Accuracy: 0.8642874484098082
```

	precision	recall	f1-score	support
0	0.94	0.90	0.92	3655
1	0.42	0.56	0.48	464
accuracy			0.86	4119
macro avg	0.68	0.73	0.70	4119
weighted avg	0.88	0.86	0.87	4119

```
Predicted probabilities (first 5):
[[1.00000000e+000 2.85474723e-142]
 [1.00000000e+000 1.11627355e-188]
 [1.00000000e+000 1.19171166e-174]
 [0.00000000e+000 1.00000000e+000]
 [1.00000000e+000 1.71562671e-025]]
```

```
/Users/Laura/miniforge3/envs/my_env/lib/python3.12/site-packages/sklearn/base.py:486: UserWarning: X has
feature names, but LogisticRegression was fitted without feature names
  warnings.warn(
/Users/Laura/miniforge3/envs/my_env/lib/python3.12/site-packages/sklearn/base.py:486: UserWarning: X has
feature names, but LogisticRegression was fitted without feature names
  warnings.warn(
```

```
cm = confusion_matrix(y_val, y_val_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No (0)', 'Yes (1)'])
disp.plot(cmap='Blues')
plt.title('Confusion Matrix - Validation Set')
plt.tight_layout()
plt.show()
```

```
<Figure size 640x480 with 2 Axes>
```

Here is my confusion matrix on the validation after training my model and below my calcu

Accuracy: 0.86428744841

Precision: 0.5625

Recall: 0.42301458671

Compare the model's accuracy with the Zero Rule baseline and briefly interpret the result.

* after calculating the baseline and my model's accuracy, we can see that the model's accuracy is worse by 2% than the baseline. Therefore there is some further work to be done in the training of the model in order to make it more accurate than the maseline.

```
from collections import Counter
#calculate the zero rule baseline accuracy for the bankind dataset
df = pd.read_csv("bank-additional-full.csv", sep=";")
target = "y"

counts = Counter(df[target])
total = len(df)
majority = counts.most_common(1)[0][0]
predictions = [majority] * total
accuracy = counts[majority] / total

print(f"\nTarget column : '{target}'")

print(f"Zero Rule Accuracy : {accuracy:.4f} ({accuracy:.2%})")
```

Target column : 'y'
Zero Rule Accuracy : 0.8873 (88.73%)